

Trabajo Práctico Integrador

Gestión de Datos de Países en Python: Filtros, Ordenamientos y Estadísticas

Materia: Programación 1

Integrantes:

Ivan Mendoza

Adrián Candas

17 de Junio de 2026

Índice

1. Marco Teórico	3
1.1 Listas	3
1.2 Diccionarios	3
1.3 Funciones	4
1.4 Estructuras Condicionales	4
1.5 Ordenamientos	4
1.6 Estadísticas Básicas	4
1.7 Archivos CSV	4
2. Decisiones Técnicas y Arquitectura	5
2.1 Estructura del programa	5
2.2 Diagrama de flujo	5
2.3 Capturas de pantalla	6
3. Dificultades y Conclusiones	9
4. Bibliografía / Webgrafía	10
5. Videos Demostrativos	10
6. Link GitHub	10

1. Marco Teórico

El presente trabajo aplica los conceptos fundamentales aprendidos en Programación 1 para desarrollar un sistema de gestión de datos sobre países. A continuación se explican los conceptos teóricos que sustentan cada funcionalidad implementada.

1.1 Listas

Una lista es una estructura de datos en Python que permite almacenar una colección ordenada y mutable de elementos. Se define con corchetes `[ ]` y puede contener elementos de cualquier tipo, incluso otros objetos compuestos como diccionarios.

En este proyecto, la variable **países** es una lista que contiene todos los países del dataset. Se utiliza **.append()** para agregar nuevos países y **.copy()** para crear copias temporales durante los ordenamientos sin modificar la lista original.

1.2 Diccionarios

Un diccionario es una estructura de datos que almacena pares clave-valor. Se define con llaves `{ }` y permite acceder a cada valor mediante su clave de manera eficiente. Son mutables, lo que permite modificar sus valores en cualquier momento.

Cada país del sistema se representa con un diccionario de cuatro claves: **'nombre'** (string), **'poblacion'** (int), **'superficie'** (int) y **'continente'** (string). Esta estructura hace el código más legible e intuitivo que si se usaran listas anidadas, ya que el acceso es por nombre de campo.

1.3 Funciones

Las funciones son bloques de código reutilizables que encapsulan una tarea específica. Se definen con la palabra clave **def**, pueden recibir parámetros y retornar valores con **return**. Su uso promueve la modularización del código y facilita su lectura, prueba y mantenimiento.

El sistema aplica el principio *una función = una responsabilidad*: **cargar_paises()** se ocupa exclusivamente de leer el CSV, **guardar_paises()** de escribirlo, **agregar_pais()** del alta de un país, y así cada operación tiene su propia función delimitada.

1.4 Estructuras Condicionales

Las estructuras condicionales permiten que el programa tome distintos caminos de ejecución según si una condición se cumple o no. Python usa **if**, **elif** y **else** para definir estas ramas. También se usan dentro de bucles **while** para controlar la repetición.

En el sistema se aplican en el menú principal (para ejecutar la opción elegida), en las validaciones de entrada (para detectar errores del usuario) y en las funciones de filtrado (para determinar si cada país cumple el criterio de búsqueda).

1.5 Ordenamientos

Un algoritmo de ordenamiento reorganiza los elementos de una lista según un criterio determinado. Existen múltiples algoritmos con distintas eficiencias: burbuja, selección, inserción, quicksort, entre otros.

En este proyecto se implementó el algoritmo de selección (**Selection Sort**). Opera con dos bucles anidados: el externo recorre cada posición de la lista, y el interno compara el elemento actual con todos los elementos siguientes. Cuando se detecta que deben intercambiarse, se usa una variable temporal **temp** para realizar el swap. Se trabaja siempre sobre una copia (**.copy()**) para no alterar la lista original.

1.6 Estadísticas Básicas

Las estadísticas básicas permiten resumir y describir numéricamente un conjunto de datos. El sistema implementa los siguientes indicadores:

Máximo y mínimo de población: se recorre la lista manteniendo un "ganador" actualizado. Cada elemento se compara con el actual máximo o mínimo, y se reemplaza si corresponde.

Promedio de población y superficie: se acumula la suma de todos los valores del campo y se divide por la cantidad total de países.

Conteo por continente: se recorre la lista con variables contadoras para cada continente, incrementando el contador correspondiente en cada iteración.

1.7 Archivos CSV

CSV (*Comma-Separated Values*) es un formato de archivo de texto plano en el que cada línea representa un registro y los campos se separan por comas. La primera línea suele contener los encabezados de columna. Es un formato ampliamente utilizado para el intercambio de datos tabulares entre sistemas.

En Python se accede a archivos con la función **open()** dentro de un bloque **with**, que garantiza el cierre automático del archivo. La primera línea (encabezado) se omite con **next(archivo)**. Cada línea se procesa con **.strip()** para eliminar saltos de línea y **.split(",")** para separar los campos. Para escritura se abre el archivo en modo **"w"** y se escriben los datos línea por línea con **archivo.write()**.

2. Decisiones Técnicas y Arquitectura

2.1 Estructura del programa

El programa se organizó en bloques funcionales bien diferenciados. Las principales decisiones de diseño fueron las siguientes:

Constante ARCHIVO_CSV: La ruta del archivo CSV se define como una constante al inicio del programa. Esto centraliza su configuración: si la ruta cambia, solo hay que modificarla en un lugar.

Funciones de validación reutilizables: Se implementaron `pedir_texto()` y `pedir_entero()` para evitar código duplicado en cada función que solicite datos al usuario. `pedir_texto()` garantiza que el campo no esté vacío; `pedir_entero()` garantiza que sea un número entero mayor a cero.

Carga única al inicio del programa: El archivo CSV se lee una sola vez al iniciar mediante `cargar_paises()`. La lista resultante se pasa como parámetro a todas las funciones que la necesitan. Esto evita lecturas repetidas del disco y mantiene la consistencia de los datos en memoria.

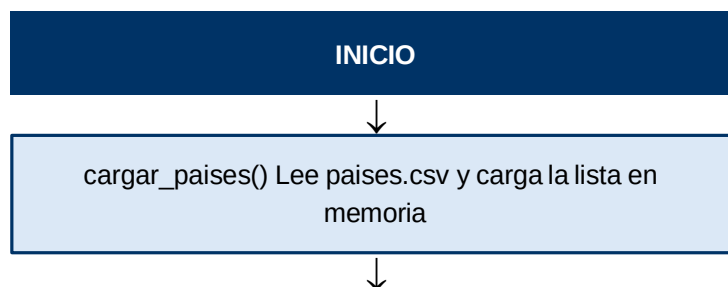
Escritura completa al modificar: Cada vez que se agrega o actualiza un país, `guardar_paises()` reescribe el archivo completo con la lista actualizada. Esto garantiza que el CSV siempre refleje el estado actual de los datos.

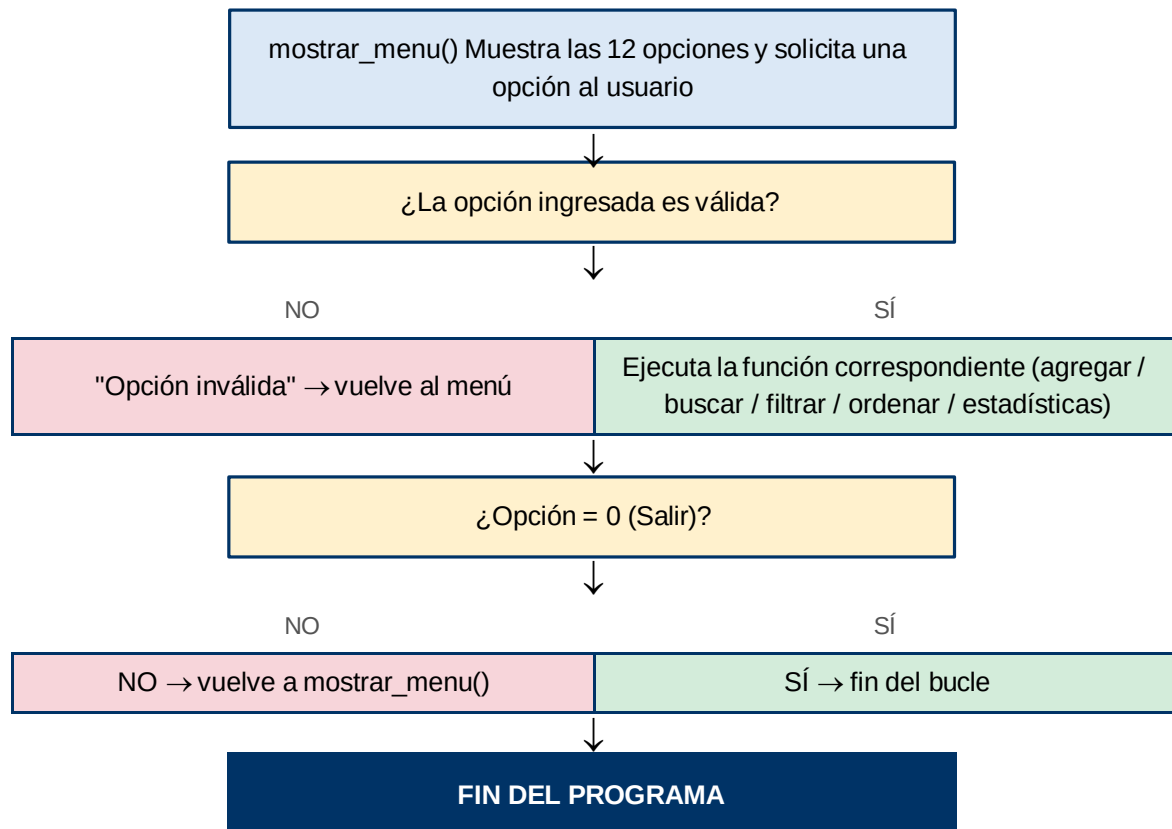
Ordenamiento sin modificar la lista original: Las funciones de ordenamiento trabajan sobre una copia de la lista (`.copy()`) para preservar el orden original. Esto permite ordenar por distintos criterios de forma independiente.

Lectura manual del CSV sin módulos externos: Se optó por leer el CSV con `open()` y `split()` en lugar de usar el módulo `csv`, para mantener coherencia con los conceptos vistos en la cursada y no depender de librerías externas para una operación sencilla.

2.2 Diagrama de flujo del programa

El siguiente diagrama representa el flujo principal de ejecución del sistema:





2.3 Capturas de Pantalla

Agregar país

```

===== GESTIÓN DE PAÍSES =====
1. Mostrar países
2. Agregar un país
3. Actualizar un país
4. Buscar por nombre
5. Filtrar por continente
6. Filtrar por rango de población
7. Filtrar por rango de superficie
8. Ordenar por nombre
9. Ordenar por población
10. Ordenar por superficie ascendente
11. Ordenar por superficie descendente
12. Mostrar estadísticas
0. Salir
Ingrese una opción: 2

Ingrese el nombre del país: Rumania
Ingrese la población: 28500200
Ingrese la superficie en km²: 32000
Ingrese el continente: Europa

País agregado correctamente.
  
```

Buscar país

```
===== GESTIÓN DE PAÍSES =====
1.  Mostrar países
2.  Agregar un país
3.  Actualizar un país
4.  Buscar por nombre
5.  Filtrar por continente
6.  Filtrar por rango de población
7.  Filtrar por rango de superficie
8.  Ordenar por nombre
9.  Ordenar por población
10. Ordenar por superficie ascendente
11. Ordenar por superficie descendente
12. Mostrar estadísticas
0.  Salir
Ingrese una opción: 4
```

Ingrese el nombre o parte del nombre del país: Brasil

```
-----
Nombre:      Brasil
Población:   213,993,437
Superficie:  8,515,767 km²
Continente:  America
-----
```

Filtrar por Continente

```
===== GESTIÓN DE PAÍSES =====
1.  Mostrar países
2.  Agregar un país
3.  Actualizar un país
4.  Buscar por nombre
5.  Filtrar por continente
6.  Filtrar por rango de población
7.  Filtrar por rango de superficie
8.  Ordenar por nombre
9.  Ordenar por población
10. Ordenar por superficie ascendente
11. Ordenar por superficie descendente
12. Mostrar estadísticas
0.  Salir
Ingrese una opción: 5
Ingrese continente: africa
```

```

-----
Nombre:      Nigeria
Población:   206,139,589
Superficie:  923,768 km²
Continente:  Africa
-----

Nombre:      Etiopía
Población:   114,963,588
Superficie:  1,104,300 km²
Continente:  Africa
-----

Nombre:      Egipto
Población:   102,334,404
Superficie:  1,001,449 km²
Continente:  Africa
-----

Nombre:      Republica Democratica del Congo
Población:   89,561,403
Superficie:  2,344,858 km²
Continente:  Africa
-----

Nombre:      Tanzania
Población:   59,734,218
Superficie:  945,087 km²
Continente:  Africa
-----

Nombre:      Kenia
Población:   53,771,296
Superficie:  580,367 km²
Continente:  Africa
-----

Nombre:      Sudafrica
Población:   59.308.690

```

Actualizar datos existentes

```

===== GESTIÓN DE PAÍSES =====
1.  Mostrar países
2.  Agregar un país
3.  Actualizar un país
4.  Buscar por nombre
5.  Filtrar por continente
6.  Filtrar por rango de población
7.  Filtrar por rango de superficie
8.  Ordenar por nombre
9.  Ordenar por población
10. Ordenar por superficie ascendente
11. Ordenar por superficie descendente
12. Mostrar estadísticas
0.  Salir
Ingrese una opción: 3

```

Ingrese el nombre exacto del país a actualizar: Argentina

```

País encontrado:
Nombre: Argentina
Población actual: 45376763
Superficie actual: 2780400
Ingrese la nueva población: 46820150
Ingrese la nueva superficie en km²: 2780400

```

País actualizado correctamente.

3. Dificultades y Conclusiones

Dificultades encontradas

Durante el desarrollo surgieron distintos obstáculos técnicos que requirieron análisis y búsqueda de soluciones:

Manejo de caracteres especiales: Se evaluó incluir tildes en los nombres de los continentes (América, África, Oceanía), pero esto generaba riesgo de inconsistencias al comparar cadenas si el usuario las ingresaba sin tilde o con distinta codificación. Se decidió estandarizar el dataset y las comparaciones sin tildes (America, Europa, Asia, Africa, Oceania), manteniendo además la lectura y escritura del archivo en codificación UTF-8 para evitar problemas con el resto de los caracteres especiales del texto.

Persistencia de datos: Fue necesario entender cómo sincronizar los datos en memoria con el archivo CSV. Se optó por reescribir el archivo completo (`guardar_paises()`) cada vez que se produce un cambio, lo que garantiza consistencia entre la lista en memoria y el archivo en disco.

Validaciones de entrada: Fue necesario diseñar funciones robustas que no fallen ante entradas incorrectas del usuario, como texto donde se espera un número o campos vacíos. Se implementaron `pedir_texto()` y `pedir_entero()` con bucles `while` que repiten la solicitud hasta obtener un dato válido.

Integración del trabajo en equipo: Coordinar las partes desarrolladas por cada integrante requirió establecer acuerdos sobre el nombre de las funciones (convención `snake_case`), el formato de los datos y la estructura general del programa para lograr una integración coherente.

Conclusiones

El desarrollo de este trabajo práctico permitió consolidar los conceptos fundamentales de Programación 1 aplicándolos a un problema concreto y completo. La experiencia de diseñar un sistema que incluye lectura y escritura de archivos, validaciones, múltiples operaciones sobre datos y una interfaz de menú resultó muy formativa.

Comprendimos en la práctica la importancia de modularizar el código: las funciones pequeñas y bien definidas son mucho más fáciles de depurar y extender que bloques grandes. También experimentamos el valor de elegir las estructuras de datos correctas: representar cada país como un diccionario hizo que el acceso a sus atributos fuera natural e intuitivo.

El trabajo en equipo nos enseñó la necesidad de acordar el diseño antes de programar, para evitar incompatibilidades al integrar las partes. La adopción de la convención `snake_case` y la modularización en funciones facilitó esa integración. En conjunto, este proyecto representó un paso importante en nuestra formación como programadores.

4. Bibliografía / Webgrafía

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press. Enlace: <https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/>

Python Software Foundation. (2024). *Built-in Types — Lists*. Python 3 Documentation. Enlace: <https://docs.python.org/3/library/stdtypes.html#lists>

Python Software Foundation. (2024). *Built-in Types — Dictionaries*. Python 3 Documentation. Enlace: <https://docs.python.org/3/library/stdtypes.html#mapping-types-dict>

Python Software Foundation. (2024). *Defining Functions*. Python 3 Documentation. Enlace: <https://docs.python.org/3/tutorial/controlflow.html#defining-functions>

Python Software Foundation. (2024). *More Control Flow Tools*. Python 3 Documentation. Enlace: <https://docs.python.org/3/tutorial/controlflow.html>

Python Software Foundation. (2024). *Reading and Writing Files*. Python 3 Documentation. Enlace: <https://docs.python.org/3/tutorial/inputoutput.html>

Real Python. (2024). *Dictionaries in Python*. Enlace: <https://realpython.com/python-dicts/>

Real Python. (2024). *Lists and Tuples in Python*. Enlace: <https://realpython.com/python-lists-tuples/>

Real Python. (2024). *Python CSV: Reading and Writing CSV Files*. Enlace: <https://realpython.com/python-csv/>

Wentworth, P., Elkner, J., Downey, A., & Meyers, C. (2012). *How to Think Like a Computer Scientist: Learning with Python 3*. OpenBookProject. Enlace: <https://openbookproject.net/thinkcs/python/english3e/>

5. Videos Demostrativos

Ivan Mendoza: <https://www.youtube.com/watch?v=-blHZxb0U5w>

Adrián Candas: https://drive.google.com/file/d/11R4x_JApaEUQnRQurBAL3bUZzk7fCBHI/view

6. Link GitHub

Link: https://github.com/IvanAM07/Trabajo_Practico_Integrador.py